

DENTAL CONNECT

PRODUCTION MASTER PRODUCT REQUIREMENTS DOCUMENT

Real production architecture for a secure, fast, modular and future-expandable dental ecosystem.

Document version	2.1 - Clinic-owned Patient & Restricted Marketplace Edition
Initial market	Dar es Salaam, Tanzania
Initial delivery	Responsive web application + versioned REST API
Architecture	Laravel modular monolith with module fault isolation
Database	MariaDB / InnoDB - Hostinger compatible
Status	Build specification - adjusted product model approved 08 Sep 2026

Production rule

Fast response, security and fault containment are release requirements equal to business functionality. A feature is not complete if it works only under ideal conditions.

Prepared as the definitive requirements baseline for implementation, testing, deployment, operations and future ecosystem expansion.

0. Document Control & Source of Truth

Item	Definition
Product	Dental Connect
Document purpose	Define the real production system, module boundaries, data model, performance, security, hosting, API and launch requirements.
Primary business source	Client-provided 7-page DENTAL ECOSYSTEM brief.
UI source	Approved Dental Connect visual direction, attached mobile healthcare references, and mockup_manifest.csv.
Earlier planning source	Dental_Ecosystem_Master_PRD.pdf and Production Master PRD v2.0. This v2.1 updates patient ownership, marketplace access, branding and UI direction.
Technical validation	Official Hostinger documentation and Laravel 13 support documentation reviewed September 2026.
Initial deployment	Hostinger Web/Cloud compatible; Cloud class preferred if resource budget permits.
Future deployment	VPS/cloud when native apps, Redis, heavy queues, clinical records or sustained traffic justify it.
Production domain	dentalconnect.co.tz

Interpretation rule

Every future capability requested by the client remains part of the product roadmap. Phase 1 defines what is built now; architecture readiness defines what must not be blocked later.

0.1 Source-of-truth hierarchy

- 1. Signed commercial agreement / approved change orders.
- 2. This Production Master PRD v2.0 for product, architecture, data, security and acceptance.
- 3. Approved UI/UX mockups for layout and interaction intent.
- 4. Original client brief for requested business vision and future ecosystem coverage.
- 5. Implementation tickets and sprint acceptance criteria that do not conflict with higher sources.

0.2 Requirement notation

Requirements use IDs such as ARCH-001, DB-004, SEC-010 and MOD-APT-003. P0 means launch-blocking; P1 means required for production quality but may be delivered within the agreed stabilization window; P2 means future expansion or optimization.

1. Executive Product Definition

Dental Connect is a multi-sided dental-industry ecosystem connecting patients, verified dental clinics, dental professionals, dental-material suppliers and a central platform operator. The initial production release is a responsive web platform, not a disposable prototype: it must be secure, API-ready, database-backed, auditable and structurally ready for native applications and larger ecosystem modules. The dental-material marketplace is a controlled business workspace and is not exposed to public visitors or patient accounts.

The client requires patient registration/login, clinic registration and verification, clinic discovery, clinic profiles, appointment requests, clinic dashboards, clinic-owned patient management, supplier registration, a restricted dental-products marketplace for clinics/suppliers/authorized administrators, administration, notifications, analytics, a secure database and role-based access control. Future expansion includes mobile applications, payments, ordering, delivery, financing, insurance, cloud dental records, AI, corporate programs, loyalty and national/regional growth.

1.1 MVP product loop

Core loop

Patient discovers verified clinic -> patient requests appointment -> clinic manages request -> clinic accesses dental marketplace -> supplier receives enquiry/RFQ -> platform administration verifies and governs all actors -> analytics measure demand and operations.

1.2 Production objectives

- Launch a real commercial web platform suitable for controlled Dar es Salaam market testing.
- Keep ordinary user actions fast and predictable on mobile connections.
- Protect identity, business-verification documents and platform administration with server-side authorization.
- Prevent one optional module or external provider from making unrelated modules unusable.
- Use a Hostinger-compatible SQL architecture that can later move to VPS/cloud without data-model replacement.
- Keep code ownership, hosting, repositories and production accounts under the client company.

2. Architecture Decision Record

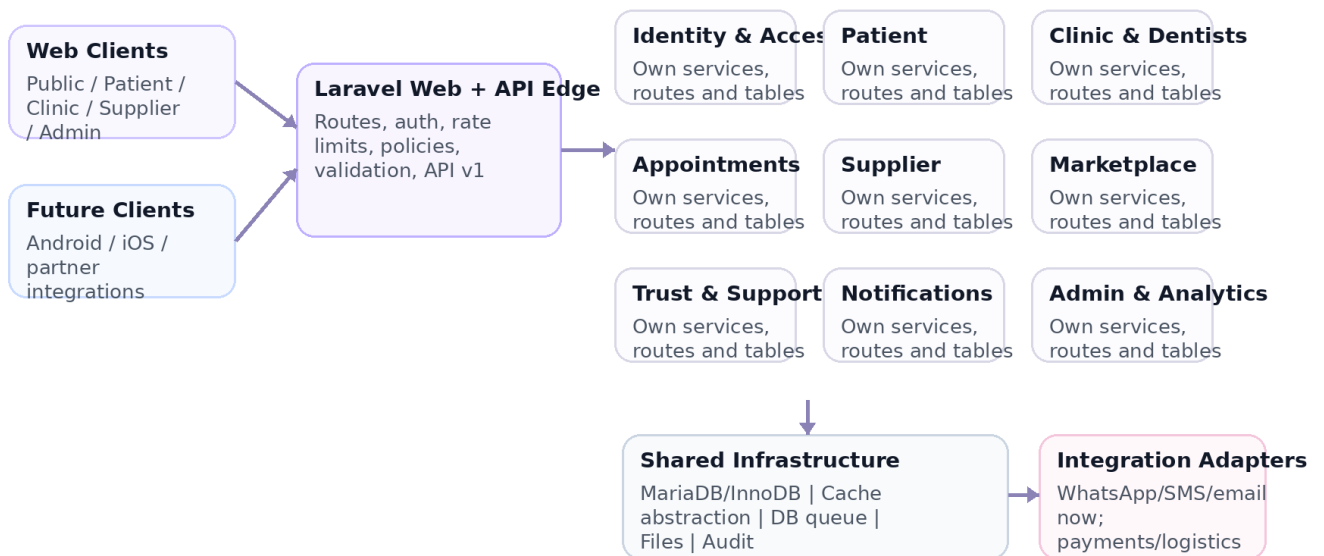
2.1 Decision: modular monolith for Phase 1

The platform SHALL be implemented as one Laravel application and one primary relational database, divided into explicit business modules with controlled interfaces. This is intentionally simpler than microservices while preserving a migration path to independently deployed services later.

Why not microservices now?

Microservices would add network calls, service discovery, distributed transactions, more servers, more monitoring, higher cost and a larger security surface before the product has validated demand. A disciplined modular monolith delivers most of the maintainability benefit at MVP scale with far less operational risk.

Production Architecture - Modular Monolith with Fault Isolation



Principle: one deployable application for MVP simplicity; module failures and external-provider outages must degrade locally, not cascade. Shared host/database outage can still affect all modules; true process-level independence is a future service-extraction option on VP.

Figure 1. Phase 1 production architecture and fault-containment posture.

2.2 Fault-isolation reality

Module-wise design does not mean that a single shared-host deployment can guarantee independent uptime if the web server or primary database itself fails. The Phase 1 requirement is application-level isolation: a failing marketplace integration, SMS provider, analytics job or supplier route must not crash patient login, clinic appointments or administration. True process-level independent uptime is achieved later by extracting selected modules onto separate services/VPS infrastructure when justified.

ID	Priority	Requirement
ARCH-001	P0	One deployable Laravel application with explicit domain modules.
ARCH-002	P0	Each module owns its routes, controllers, services, policies, validation, migrations/data ownership documentation and tests.
ARCH-003	P0	Cross-module writes occur through application services/contracts or domain events, not arbitrary model manipulation from unrelated modules.
ARCH-004	P0	External integrations are accessed only through adapters with timeout, retry and failure handling.
ARCH-005	P0	Any non-core module can be disabled by feature flag/configuration without blocking authentication or administration.
ARCH-006	P0	No module performs remote API calls during application boot.
ARCH-007	P0	Module exceptions are converted into recoverable error responses; no white-screen/blank-page failure.
ARCH-008	P1	Health checks report core application, database, queues and integration status independently.
ARCH-009	P2	High-growth modules can be extracted into services while preserving API contracts and public IDs.

3. Recommended Technology Stack

Layer	Production choice	Reason
Language	PHP 8.3 or 8.4	Supported by Hostinger; modern security/performance baseline and compatible with Laravel 13.
Framework	Laravel 13	Current supported Laravel generation in 2026; mature routing, security, ORM, queues, cache, jobs, policies, testing and API support.
Web UI	Blade + Livewire + Alpine.js + Tailwind CSS	Single-codebase, low JavaScript overhead, Hostinger-friendly and fast to maintain while still providing app-like interactions.
API	Laravel REST API under /api/v1	Reuses the same domain/application services and supports future Android/iOS or partners.
Database	MariaDB + InnoDB	Hostinger Web/Cloud uses MariaDB; InnoDB is supported and provides transactions, row locking and foreign keys.
Cache	Laravel Cache abstraction; file/database initially; Redis when on VPS	Avoids a VPS-only dependency in Phase 1 while keeping cache backend replaceable.
Queues	Database queue + cron-driven workers initially; Redis/Horizon on VPS later	Works with Hostinger Web/Cloud cron; isolates slow notifications and jobs from web requests.
Files	Private local storage outside public web root initially; S3-compatible object storage later	Verification documents remain protected and storage can migrate without business-logic rewrite.
Charts	Chart.js loaded only on analytics screens	Small, established, sufficient for MVP analytics.
Repository	Company-controlled GitHub repository	Full source ownership, traceability and portability.

3.1 Package policy

- Prefer Laravel first-party capabilities before introducing third-party packages.
- Every production dependency must have an active maintenance history, compatible license and documented purpose.
- A package may not control the core data model in a way that creates lock-in.
- RBAC may use a mature MIT-licensed package such as spatie/laravel-permission, but contextual clinic/supplier scope must still be enforced by application policies.
- No large front-end framework or analytics library may be loaded globally for a feature used on one route.

4. Application Module Architecture

The codebase should use a domain-first folder structure without requiring a heavy module framework. Laravel service providers may register each module, but the application remains understandable to any competent Laravel developer.

Recommended structure:

```
app/  
  Modules/  
    Identity/  
      Domain/  
      Application/  
      Http/  
      Infrastructure/  
      Database/  
      Tests/  
    Patient/  
    Clinic/  
    Appointment/  
    Supplier/  
    Marketplace/  
    Notification/  
    TrustSupport/  
    AdminAnalytics/  
  Shared/  
    Contracts/  
    Support/  
    ValueObjects/  
  routes/  
    web.php  
    api.php
```

4.1 Module communication rules

- A module may read another module through a defined query service or stable read contract when necessary.
- A module may request another module to change state only through an application service/command or domain event.
- Direct cross-module table writes are prohibited.
- Events used for non-critical side effects (notifications, analytics, search indexing) are queued and retryable.
- Critical same-database state changes that must succeed together use one database transaction.
- External-provider failure must never roll back a valid core business transaction after it has been committed; the provider call is queued after commit where possible.
- Module route groups have explicit middleware and authorization policies.
- Feature flags can temporarily disable optional modules or integrations without code removal.

5. Module Catalogue & Ownership

Code	Module	Owns	Stage
CORE	Core Platform	Settings, feature flags, files, shared primitives, health	Phase 1
IAM	Identity & Access	Users, sessions, authentication, roles, permissions, MFA	Phase 1
PAT	Patient	Patient profile, preferences, favorites, consent	Phase 1
CLN	Clinic & Practitioner	Clinic onboarding, locations, services, dentists, verification	Phase 1
APT	Appointments	Availability, requests, status history, reschedule/cancel/complete	Phase 1
SUP	Supplier	Supplier onboarding, verification, profile	Phase 1
MKT	Marketplace	Categories, products, inventory status, RFQs/enquiries	Phase 1
NOT	Notifications	In-app, email/SMS/WhatsApp adapter orchestration, delivery logs	Phase 1
TRU	Trust & Support	Reviews, reports, complaints/disputes	Phase 1 baseline
ADM	Administration	Governance, verification queues, moderation, settings	Phase 1
ANL	Analytics & Reporting	Operational metrics, aggregated dashboards, exports	Phase 1 basic

Code	Module	Owns	Stage
AUD	Audit	Append-only privileged action/event audit	Phase 1
PAY	Payments & Commissions	Mobile money/cards, payments, commissions, settlement	Future
ORD	Orders	Cart, order, order items, order lifecycle	Future
LOG	Delivery & Logistics	Shipment, dispatch, delivery tracking	Future
FIN	Patient Financing	Applications, offers, installments, financing partners	Future
INS	Insurance	Providers, eligibility, claims	Future
REC	Clinical Records	Dental records, treatment data, clinical permissions	Future - isolated
COR	Corporate Programs	Employers, employee benefits, reporting	Future
LOY	Loyalty & Membership	Plans, benefits, points/rewards	Future
AI	AI Assistance	Support/search/operations assistance with safety controls	Future

6. Roles, Access & Data Isolation

Role	Primary scope	Key restrictions
Guest	Public landing and verified clinic directory/profile only	No marketplace, private patient data, clinic internals or supplier workspace.
Patient	Own login identity plus clinic-specific patient enrollment(s), appointments and permitted clinic-scoped records	No marketplace. Cannot access another patient or another clinic relationship unless explicitly enrolled and authorized.
Clinic Owner/Admin	Assigned clinic(s), clinic patients, appointments, services/dentists, analytics and marketplace	Cannot access unrelated clinics or their patients; verification files limited by permission.
Clinic Staff	Delegated clinic functions including permitted patient management and marketplace	Permission bundle restricts patient, profile, dentist, appointment, marketplace and analytics actions.
Supplier Owner/Admin	Own supplier organization, products, RFQs and restricted marketplace	Cannot access unrelated supplier private records; can view marketplace catalogue according to marketplace policy.
Supplier Staff	Delegated supplier product/RFQ/marketplace functions	Limited by supplier permission bundle and organization scope.
Platform Operations	Assigned admin permissions	No automatic full access; least privilege.
Super Admin	System governance	MFA mandatory; all privileged actions audited.

ID	Priority	Requirement
IAM-001	P0	All authorization SHALL be enforced server-side. Hiding UI controls is never sufficient.
IAM-002	P0	Clinic and supplier queries SHALL always enforce organization scope.
IAM-003	P0	Suspended users are blocked from protected mutations and see a controlled status page.
IAM-004	P0	Super Admin and privileged admin roles SHALL use MFA before production launch.
IAM-005	P0	Passwords are never reversible or visible to administrators.
IAM-006	P1	Permission changes are audited with actor, target, before/after state and timestamp.
IAM-007	P1	Session invalidation is supported for password reset, suspension and security events.

7. Phase 1 Functional Requirements by Module

7.1 Identity & Access

- Patient, clinic and supplier registration flows are distinct but reuse the same secure identity system. A patient's login identity may be reused, but each clinic creates and owns a separate clinic-patient enrollment/profile for that person.
- Login, logout, password reset, email/phone verification when enabled, session management and account status handling.
- Role and permission assignment controlled by authorized administrators.
- Admin MFA required; patient MFA may be offered later.
- Future API/mobile tokens use Laravel Sanctum-compatible authentication or equivalent without replacing user identities.

7.2 Patient module

- Patient identity/profile data is kept minimal at platform level; the operational patient record belongs to a specific clinic through `clinic_patients`.
- Patient can discover verified clinics by name, location, service and specialty before enrolling/registering with a clinic.
- If a patient chooses a different clinic, the patient must complete a new clinic enrollment/registration for that clinic; the same login identity may be reused where securely matched.
- Patient sees appointments, records and communication only within the active clinic-patient relationship; cross-clinic clinical data is not exposed by default.
- Notification center and communication entry points are scoped to the relevant clinic-patient relationship.
- Review submission is linked to the clinic-patient relationship and an eligible completed appointment.
- Privacy/consent records store clinic context, policy version and timestamp where consent is clinic-specific.

7.3 Clinic & Practitioner module

- Clinic registration and structured onboarding.
- Clinic profile: public name, description, contacts, locations, opening hours, images and verification badge.
- Configurable verification document types and secure private uploads.
- Verification lifecycle: Draft, Submitted, Under Review, Changes Requested, Approved, Rejected, Suspended.
- Service and specialty selection from controlled platform catalogues.
- Dentist profiles, specialties, status and clinic association.
- Basic availability, working hours and blackout dates.
- Clinic-level analytics sourced from real appointments, enrolled patients and profile activity.
- Clinic has a My Patients module listing only patients enrolled with that clinic, with search, status, last visit, next appointment and assigned dentist where applicable.
- Clinic patient detail provides clinic-scoped demographics, appointment history, treatment/record links, notes/status and communication. It must never reveal another clinic's private patient relationship.

7.4 Appointment module

- Appointment-request model for MVP rather than complex instant chair/resource booking.
- Patient selects clinic, optional service/dentist, preferred date/time and optional note.
- Clinic can confirm, propose reschedule, decline, cancel, complete or mark no-show according to permission and state.
- Every status change is preserved in immutable status history.
- Conflicting confirmed bookings for the same dentist/time window are prevented where dentist-specific scheduling is active.
- Appointment business transaction commits before any external notification is attempted.
- Notifications are queued after commit so provider outage cannot lose the appointment.

7.5 Supplier module

- Supplier registration, profile and verification.
- Secure verification-document handling.
- Product catalogue ownership and active/inactive controls.
- Basic supplier dashboard with product/RFQ activity.
- Marketplace browsing is restricted to authenticated clinic users, supplier users and authorized platform administrators; it is not available to guests or patient accounts.

Authorized marketplace users can browse by category, keyword, supplier and availability status and view product details with images, description, brand/reference, price when shown, specifications and supplier identity.

- Phase 1 uses RFQ/enquiry instead of mandatory cart/checkout.

- Clinic users can request quotation with product, quantity and message.
- Supplier users receive RFQs and can update response/status; suppliers may also browse the marketplace to understand visible catalogue presentation.
- Authorized administrators can moderate/unpublish product listings and inspect marketplace activity.
- Supplier receives RFQ and can update response/status.
- Admin can moderate/unpublish product listings.
- Architecture reserves product/order/payment identifiers so Phase 2 commerce does not require replacing product records.

7.7 Notification module

- In-app notifications are the minimum reliable channel for system events.
- Email/SMS/WhatsApp are provider adapters, not embedded business logic.
- Appointment submitted, confirmed, rescheduled, declined, cancelled and reminder events are supported.
- Delivery attempts record provider, status, retry count, reference and error category where available.
- Provider timeouts, rate limits or downtime do not break appointment, clinic or marketplace transactions.

7.8 Trust, reviews, complaints & disputes

- Patient review/rating baseline with moderation and report capability.
- Complaint/support record with category, subject, description, status, requester and internal notes.
- Dispute model remains simple before paid orders exist, but data model supports future transaction/order linkage.
- Admin moderation actions and status changes are audited.

7.9 Administration & analytics

- Central dashboard shows real counts and trends; no production demo values.
- Clinic and supplier verification queues.
- Patient, clinic, supplier, dentist, appointment, product, review and complaint management.
- Roles and permissions management.
- Basic operational reports and CSV export where needed.
- Audit log view restricted to authorized administrators.
- Analytics queries use aggregated tables/cache where direct transactional scans would be expensive.

8. Web & API Architecture

Surface	Route convention	Purpose
Public	/	Landing, clinic directory/profile, role selection and public information. No marketplace.
Patient	/app/*	Clinic-scoped patient app: dashboard, appointments, permitted records, profile, notifications. No marketplace.
Clinic	/clinic/*	Clinic onboarding, My Patients, appointments, dentists, profile, analytics and marketplace.
Supplier	/supplier/*	Supplier onboarding, products, RFQs, marketplace and analytics.
Admin	/admin/*	Verification, governance, marketplace moderation, patient/clinic/supplier management, reporting and settings.
API	/api/v1/*	Future mobile clients and approved integrations; same application services.
Health	/health and internal module checks	Operational health without exposing secrets.

8.1 API rules

- JSON API responses use consistent success, validation and error envelopes.
- All list endpoints are paginated; unbounded collection responses are prohibited.

- Public IDs use ULIDs/opaque identifiers rather than exposing sequential database keys.
- API versioning starts at /api/v1 and incompatible changes require a new version or documented compatibility layer.
- Rate limits apply by route category, authenticated user and IP as appropriate.
- Idempotency keys are required for future payment/order endpoints and recommended for any externally retried critical mutation.
- API authorization uses the same policies/services as web controllers; business rules must not be duplicated in controllers.

9. Database Architecture - MariaDB/InnoDB

Hostinger Web and Cloud hosting use MariaDB and support InnoDB. The Phase 1 database SHALL therefore target MariaDB/InnoDB directly, avoiding PostgreSQL- or Redis-only features until the platform is moved to VPS/cloud infrastructure.

Database principle

Optimize for the actual query paths the application will execute. Every index has a purpose; every heavy endpoint has a query plan; no dashboard may scan entire transactional tables on every page load.

9.1 Key strategy

- Internal primary key: BIGINT UNSIGNED AUTO_INCREMENT for compact, fast clustered indexing in InnoDB.
- Public identifier: ULID stored as CHAR(26) ASCII with a UNIQUE index on externally addressable records.
- Internal BIGINT keys are never exposed as the only public identifier in URLs/APIs.
- Foreign keys use matching unsigned types and explicit indexes.
- Hard deletes are avoided for transactional/audit entities; lifecycle status or soft delete is used where business/legal retention requires it.
- Created/updated timestamps use UTC in database; display converts to the user/organization timezone.

9.2 Table design rules

- Use the smallest appropriate numeric and string types; avoid TEXT where VARCHAR or coded reference is sufficient.
- Avoid large JSON blobs for relational business data. JSON is permitted for provider payload snapshots, flexible metadata or non-query-critical configuration.
- No comma-separated IDs; use junction tables.
- No duplicated names/labels where a stable foreign key should reference a catalogue.
- Use NOT NULL wherever absence has no business meaning.
- Use unique constraints for business invariants such as clinic-service pairs and one review per eligible appointment/patient.
- Use DECIMAL for future monetary values; never FLOAT/DOUBLE for money.
- Use UTC datetime values and explicit status codes; do not rely on localized display strings in the database.

10. Logical Data Model & Table Ownership

Owner	Table / group	Purpose	Primary index/constraint notes
IAM	users	Identity, login, account status	PK id; UQ public_id; UQ email/phone where applicable; IDX status
IAM	roles / permissions / assignments	RBAC	Unique role/permission names; user/role composite indexes
PAT	clinic_patients	Clinic-owned patient enrollment/profile linked to platform user identity	UQ clinic_id+user_id; UQ clinic_id+patient_number; IDX clinic_id+status+created_at
PAT	patient_consents	Clinic/policy consent history	IDX clinic_patient_id, type, created_at
PAT	patient_clinic_preferences	Optional pre-enrollment/saved clinic preferences	UQ user_id+clinic_id
CLN	clinics	Clinic master	UQ public_id; UQ slug; IDX verification_status+is_active
CLN	clinic_locations	Location/address	IDX clinic_id; IDX region/city/area; lat/lng
CLN	services / specialties	Master catalogues	UQ normalized name/code; IDX active+sort_order
CLN	clinic_services	Clinic service relation	UQ clinic_id+service_id; reverse IDX

Owner	Table / group	Purpose	Primary index/constraint notes
CLN	dentists	Practitioner profile	service_id+clinic_id
CLN	clinic_hours / blackout_dates	Availability source	IDX clinic_id+status; UQ public_id
CLN	verification_submissions	Verification workflow	UQ clinic/day; IDX clinic_id+date
CORE	files	Private/public file metadata	IDX clinic_id+status+created_at
APT	appointments	Clinic patient appointment requests and lifecycle	IDX owner_type+owner_id; IDX visibility
APT	appointment_status_history	Immutable status trail	IDX clinic_id+status+preferred_date; IDX clinic_patient_id+status+preferred_date; dentist schedule index
SUP	suppliers	Supplier master	IDX appointment_id+created_at
SUP	supplier_verifications	Supplier verification	UQ public_id; IDX verification_status+is_active
MKT	product_categories	Hierarchical categories	IDX supplier_id+status+created_at
MKT	products	Supplier product catalogue	IDX parent_id+active+sort_order
MKT	product_images	Product media	See hot-index plan
MKT	rfqs	Clinic-to-supplier enquiry	IDX product_id+sort_order
MKT	rfq_messages / status_history	RFQ conversation/status	See hot-index plan
TRU	reviews	Clinic reviews	IDX rfq_id+created_at
TRU	complaints	Support/dispute cases	IDX clinic_id+status+created_at ; UQ appointment_id+clinic_patient_id
NOT	notifications	In-app notification	IDX status+priority+updated_at; IDX requester_user_id
NOT	notification_deliveries	Provider delivery audit	IDX user_id+read_at+created_at
AUD	audit_logs	Privileged/system audit	IDX notification_id; IDX provider+status+created_at
ANL	analytics_events	Minimal event stream	See audit-index plan; append-oriented
ANL	daily_metrics	Pre-aggregated dashboards	IDX event_name+created_at; actor/entity indexes
CORE	system_settings / feature_flags	Runtime configuration	UQ metric_date+metric_code+scope keys
			UQ key; cache aggressively

10.1 Future module tables reserved by design, not created prematurely

Module	Future entities
PAY	payments, payment_attempts, refunds, commissions, settlements
ORD	orders, order_items, order_status_history
LOG	shipments, delivery_providers, shipment_events
FIN	financing_applications, offers, agreements, installments
INS	insurance_providers, plans, eligibility_checks, claims
REC	clinical_records, treatment_plans, clinical_documents, clinical_audit
COR	corporate_accounts, employees/beneficiaries, benefit_plans, utilization
LOY	membership_plans, memberships, points_ledger, benefits
AI	ai_sessions, safety_events, consent/usage metadata - no raw sensitive prompts without policy

11. Hot Query & Index Strategy

Indexes SHALL be defined from real access patterns and validated with EXPLAIN/EXPLAIN ANALYZE in staging with production-like data. The following are minimum starting indexes; implementation may refine column order based on measured selectivity.

Table	Query	Starting index
appointments	Clinic work queue	(clinic_id, status, preferred_date, preferred_time)
appointments	Clinic-patient appointment list	(clinic_patient_id, status, preferred_date)
appointments	Dentist schedule/conflict	(dentist_id, preferred_date, preferred_time, status)

Table	Query	Starting index
appointments	Admin monitoring	(status, created_at) and (clinic_id, created_at)
clinics	Public verified directory	(verification_status, is_active, primary_region_id) plus slug/public_id unique
clinic_services	Find clinics by service	(service_id, clinic_id) plus unique reverse pair
dentists	Clinic practitioner list	(clinic_id, status, sort_order)
products	Supplier product manager	(supplier_id, status, updated_at)
products	Marketplace category browse	(category_id, status, created_at)
products	Public product lookup	public_id unique; normalized search fields/fulltext when needed
rfqs	Supplier inbox	(supplier_id, status, created_at)
rfqs	Clinic sent enquiries	(clinic_id, status, created_at)
reviews	Clinic reviews	(clinic_id, moderation_status, created_at)
notifications	Unread notification list	(user_id, read_at, created_at)
audit_logs	Entity history	(entity_type, entity_id, created_at)
audit_logs	Actor history	(actor_user_id, created_at)
complaints	Operations queue	(status, priority, updated_at)
clinic_patients	Clinic patient directory	(clinic_id, status, last_name, first_name) plus UQ clinic_id+user_id
clinic_patients	Patient number lookup	UQ (clinic_id, patient_number)

11.1 Search policy

- Exact IDs, slugs, email/phone and codes use indexed equality queries.
- Directory filtering uses normalized foreign keys (location/service/specialty), not text matching against profile paragraphs.
- Leading-wildcard LIKE queries such as %term% are prohibited on large hot tables unless measured and justified.
- MariaDB FULLTEXT may be introduced for product/clinic keyword search when data volume requires it; Phase 1 can begin with indexed prefix search and controlled fields.
- Geographic distance search can start with stored latitude/longitude and bounded-area filtering; advanced spatial search is a later optimization if needed.

11.2 Query budget & code rules

- No N+1 ORM patterns. Controllers/services SHALL eager-load only required relationships.
- SELECT only columns required by the response; avoid SELECT * on high-volume lists.
- All operational lists use server-side pagination. Cursor pagination is preferred for very large time-ordered tables.
- Dashboard counts SHALL use cached or pre-aggregated metrics when repeated full-table COUNT/SUM would be costly.
- Reports over large ranges run as background jobs and produce downloadable files instead of blocking normal requests.
- Any query consistently exceeding 100 ms on production-like data requires review; any endpoint with repeated slow queries fails performance acceptance until justified/fixed.

12. Cache, Queue & Background Processing

12.1 Cache strategy

- Use Laravel Cache abstraction so storage can change without business-code changes.
- Cache master catalogues, system settings, public clinic profile fragments and non-sensitive directory results with short TTLs where safe.
- Do not cache authorization decisions across users in a way that can leak access.
- Invalidate clinic/product cache after approved profile/product changes.
- Use route/config/view/opcode caching in production.
- When moved to VPS, switch cache/session/queue to Redis without redesigning modules.

12.2 Queue strategy on Hostinger Web/Cloud

- Use Laravel database queue for slow/non-critical work during Phase 1.
- Hostinger cron invokes the Laravel scheduler; scheduled short queue workers process jobs with stop-when-empty/max-time behavior rather than relying on an unsupported permanent daemon.
- Queue categories: notifications, analytics aggregation, exports, cleanup/maintenance and integration webhooks.
- Failed jobs are retained, visible to operations and retryable.
- Jobs are idempotent where retry could otherwise create duplicate notifications or transactions.
- On VPS, move to Redis + Horizon/Supervisor for low-latency continuous workers.

13. Module Reliability & Failure Containment

Failure	Required behavior
SMS/WhatsApp/email provider unavailable	Appointment/RFQ still saves; notification job retries; user sees in-app confirmation.
Marketplace module throws recoverable error	Patient landing/directory/login/appointments and clinic patient management remain usable; marketplace shows controlled unavailable state to authorized roles.
Analytics aggregation fails	Transactional modules continue; stale analytics marked/alerted; job retried.
Product image/file unavailable	Product page renders placeholder/error state; no whole-page crash.
Admin export is slow	Export runs in background; admin can continue normal operations.
Payment gateway future outage	No unrelated page failure; pending payment state preserved; retries/webhook reconciliation.
Database unavailable	All DB-backed modules are affected; health check fails clearly. This is a shared infrastructure failure requiring restore/failover procedure.

ID	Priority	Reliability requirement
REL-001	P0	No major route may return a blank page because an optional integration is unavailable.
REL-002	P0	External HTTP calls use explicit connect and total timeouts; no indefinite waits.
REL-003	P0	Retries use capped exponential backoff and avoid duplicate side effects.
REL-004	P0	Critical transactions use DB transactions and after-commit jobs/events.
REL-005	P0	Module feature flags can disable malfunctioning optional functionality.
REL-006	P1	Health endpoint distinguishes application, database, queue backlog and provider status.
REL-007	P1	Operational alerts exist for repeated 5xx errors, failed jobs, backup failures and slow endpoints.

14. Security Architecture

14.1 Authentication and session security

- HTTPS required in production; HTTP redirects to HTTPS.
- Passwords use Laravel-approved one-way hashing; Argon2id preferred where confirmed by runtime, bcrypt accepted fallback.
- Secure, HttpOnly cookies with appropriate SameSite policy and HTTPS-only flag.
- CSRF protection on session-authenticated mutations.
- Login/password-reset endpoints rate-limited; repeated failures may trigger temporary lock/challenge.
- Privileged administrators require MFA and shorter idle session timeout.
- Password reset invalidates appropriate existing sessions/tokens.

14.2 Authorization and tenant-like organization isolation

- Every clinic/supplier protected query enforces organization scope on the server.
- Policies validate role, organization, assignment, ownership and record state.
- Direct URL/API manipulation to another clinic/supplier record must return 403/404 without disclosing sensitive existence details.
- Administration uses least privilege; read-only support roles must not silently gain edit rights.
- Privilege escalation paths are covered by automated negative tests.

14.3 Input, output and injection protection

- Laravel validation/Form Request classes validate all external input.
- Parameterized ORM/query-builder usage; raw SQL requires code review and bound parameters.
- Output is escaped by default; any rendered HTML from trusted administrators is sanitized and tightly controlled.
- File names supplied by users are never trusted as storage paths.
- All upload MIME/type/size checks occur server-side.

- ModSecurity/WAF should remain enabled at Hostinger unless a verified compatibility issue requires an approved exception.

14.4 API and integration security

- API rate limits by endpoint sensitivity.
- Provider credentials remain in environment/secrets configuration, never source control.
- Incoming webhooks verify provider signatures/secrets and use idempotent event references.
- Outgoing requests set timeouts and log safe metadata without leaking credentials or sensitive payloads.
- Future payment module never stores raw card PAN/CVV.

15. Privacy & Tanzania Data-Protection Readiness

The initial launch market is Dar es Salaam. Tanzania has a Personal Data Protection Act (Cap. 44 / Act No. 11 of 2022) and a Personal Data Protection Commission. Health information is treated as sensitive personal data by the regulator. The production launch therefore requires legal/product-owner review of controller/processor registration, privacy notices, lawful bases/consent, retention, data-subject rights and any cross-border data transfer arrangements. This PRD defines technical readiness, not legal advice.

- Data minimization: Phase 1 does not collect detailed diagnosis, X-rays or clinical records unless separately approved.
- Privacy notice and consent version must be stored with timestamp where consent is the basis for processing.
- Processes must support access, correction and deletion/restriction requests where legally applicable.
- Sensitive documents are private by default.
- Analytics must avoid storing more personal data than needed; use pseudonymous IDs where possible.
- Future cloud dental records require a separate clinical-data security review, stricter access, separate retention rules and likely stronger infrastructure isolation.

16. File & Verification Document Security

- Private verification documents SHALL be stored outside publicly served directories.
- Access occurs through an authorized controller/service that checks permission before streaming or issuing a short-lived link.
- Allowed MIME types and maximum sizes are configured per document type.
- Files are renamed to generated identifiers; original names are metadata only.
- Dangerous executable/script formats are rejected.
- Image/document processing occurs asynchronously where practical.
- File access and verification decisions are auditable.
- Future migration to S3-compatible object storage uses Laravel filesystem abstraction.

17. Audit Logging

Audit logs are append-oriented and separate from ordinary user activity analytics. They must answer who did what, to which protected entity, when, and from which authenticated context.

- Admin login/security events.
- Clinic/supplier approval, rejection, changes requested and suspension.
- Role/permission changes.
- Product moderation.
- Complaint/dispute status changes.
- Critical settings/feature-flag changes.
- Future payment/refund/commission actions.
- Clinical-record access when that future module is implemented.

Audit payloads must not indiscriminately copy passwords, tokens or full sensitive documents. Before/after values are captured only for fields appropriate to audit.

18. Performance & Response-Time Requirements

Performance targets are engineering targets rather than hosting-provider SLAs. They are measured on production-like staging data and representative network/device conditions.

Metric	Target / gate
Public page experience	Core content visible quickly; target LCP <= 2.5s on a representative mobile connection for optimized pages.

Metric	Target / gate
Standard server/API read	Target p95 application response $\leq 1.0s$ for common indexed reads under expected MVP concurrency.
Standard write action	Target p95 $\leq 1.5s$ excluding intentionally asynchronous external-provider completion.
Core DB query	Typical indexed query $\leq 100ms$ on production-like dataset; persistent outliers require review.
Search/filter	Target $\leq 1.5s$ for normal directory/product searches at MVP scale.
Dashboard	Initial shell/content $\leq 2s$; heavy analytics loaded from cache/aggregates or deferred.
Large export/report	Must not block normal web request; background job with completion notification.
Image/media	Responsive sizes, lazy loading and compression; no original multi-megabyte image on list cards.

18.1 Front-end speed rules

- Minimize global JavaScript; load charts/editors only on routes that need them.
- Use server-rendered HTML for first content where practical.
- Use image dimensions to prevent layout shift.
- Use optimized WebP/AVIF where supported, with sensible fallbacks.
- No autoplay background video or heavy 3D effects on operational screens.
- Use skeleton/loading states instead of blank screens for async components.

19. Logging, Monitoring & Operational Visibility

- Production debug mode disabled; stack traces not exposed to users.
- Central application log captures request correlation ID, severity, module and safe context.
- Slow-request logging records route and duration; sensitive payloads are excluded.
- Application-level database listener logs queries above a configured threshold in staging and selectively in production.
- Failed queue jobs are visible and alertable.
- External uptime monitor checks public and authenticated health endpoints as appropriate.
- Backup success/failure is monitored.
- Storage usage and database size are reviewed before Hostinger plan limits become critical.

20. Backup & Disaster Recovery

Area	MVP requirement
Database	Daily automated backup minimum; additional pre-deployment backup before risky migrations.
Files	Verification/private files included in backup or replicated to separate protected storage.
Source code	Company-controlled Git repository; protected main branch and release tags.
Environment/config	Secrets stored securely; non-secret deployment configuration documented.
Off-site copy	Maintain an encrypted copy outside the same primary hosting failure domain where commercially practical.
Restore test	Restore a production-like backup into clean staging before launch and periodically thereafter.
MVP RPO target	≤ 24 hours unless a stricter business requirement is approved.
MVP RTO target	≤ 8 hours for documented restore under normal access conditions; tighten with infrastructure upgrade if business requires.

21. Hostinger Deployment Architecture

Hostinger Web/Cloud supports Laravel/PHP, Composer, cron jobs and MariaDB. Web/Cloud plans do not provide Redis or PostgreSQL as native database services; those require VPS. The initial deployment therefore avoids requiring those services.

21.1 Recommended initial posture

- Hostinger Cloud-class hosting is preferred over the smallest shared tier if the commercial budget permits, because CPU, RAM, database connections and database-size headroom are materially better.

- Exact plan is selected at deployment based on current Hostinger pricing/limits; provider subscriptions can change.
- Use PHP 8.3/8.4 with OPcache enabled.
- Use MariaDB/InnoDB and limit connection usage; do not create unnecessary concurrent database connections.
- Use one Laravel scheduler cron entry and let the application coordinate scheduled tasks.
- Use Composer 2 and a tested deployment script/checklist.
- Production application code is deployed from the company-controlled repository or a controlled build artifact.

21.2 VPS migration triggers

- Native Android/iOS applications create sustained API concurrency beyond current shared/cloud plan headroom.
- Redis/Horizon is needed for low-latency queues, distributed cache or high notification volume.
- Database size, connections, CPU or I/O repeatedly approach hosting limits.
- Clinical-record module requires stronger isolation and operational controls.
- Background jobs or integrations require persistent workers/process supervision.
- Business requires tighter recovery/availability objectives than one shared primary database can provide.

22. Environments, Deployment & Change Safety

Environment	Purpose
Local	Developer work with non-production data.
Staging/UAT	Client review, integration testing, performance/security checks, migration rehearsal.
Production	Real users/data. Debug disabled and least-privilege access.

- No production database credentials in source control.
- Production data is not copied into development without masking and authorization.
- Database migrations are forward-safe, reversible where practical and reviewed for table locking/large-table impact.
- Back up before destructive/schema-risk deployments.
- Use maintenance-safe deployments; clear config/route/view caches only in controlled order.
- Rollback procedure exists for code release and, separately, for data migration when feasible.
- Health check and smoke test immediately after deployment.

23. Testing Strategy & Release Gates

23.1 Automated testing

- Unit tests for domain rules, status transitions and value calculations.
- Feature/API tests for registration, verification, appointments, RFQs and admin actions.
- Negative authorization tests proving one patient/clinic/supplier cannot access another party's protected data.
- Database tests for unique constraints and transaction rollback behavior.
- Integration-adapter tests for success, timeout, provider error, retry and duplicate callback.
- Critical route smoke tests after deployment.

23.2 Manual/UAT testing

- Desktop, tablet, modern Android and iPhone critical workflows.
- Current Chrome, Safari, Firefox and Edge.
- Upload validation and protected-document access.
- Empty, loading, error, permission-denied and provider-unavailable states.
- Production-like data volume for appointment lists, clinic directory, products and admin tables.
- Backup restore rehearsal before launch.

23.3 Release-blocking gates

- No blank-page failure on major routes.
- No cross-patient, cross-clinic or cross-supplier data leakage.
- Core indexes and query plans verified for production-like data.
- No unbounded production list endpoints.
- Admin MFA enabled.
- Uploads protected and validation confirmed.
- Backups configured and restore tested.
- External notification outage does not break core transactions.
- Production debug disabled and secrets absent from repository.

- Critical responsive workflows approved by product owner.

24. Development Standards

- Business rules belong in domain/application services, not Blade templates or controllers.
- Controllers are thin: authorize -> validate -> call application service -> return response.
- Eloquent models do not become cross-module global service locators.
- Every database change is a migration; manual production schema edits are prohibited except emergency procedure with documentation.
- Code style and static analysis are part of CI or pre-release checks.
- No hidden demo/test credentials in production.
- Public API changes require changelog/release-note entry.
- Complex queries include a short comment or repository method name explaining the access pattern and expected index.
- Module owner and data owner are documented for every new table.

25. Future Expansion Architecture

25.1 Native mobile applications

Android/iOS apps consume /api/v1 or later versions and share identity, permissions and domain services. The web UI is not the business-logic layer, so mobile apps do not require rebuilding appointments, verification, products or administration.

25.2 Payments, ordering and logistics

Payments and commerce are separate modules. Products remain stable catalog entities; Phase 2 adds order/payment lifecycle tables and adapter interfaces. Payment provider callbacks are idempotent and verified. Delivery is downstream of confirmed order/payment state, not embedded into marketplace browsing.

25.3 Financing and insurance

Financing applications and insurance eligibility/claims are separate bounded modules so regulatory/provider logic does not contaminate the appointment or patient-profile schema.

25.4 Cloud dental records

Clinical records are intentionally a dedicated future module with stricter access, audit, retention and data-isolation rules. Before implementation, legal/privacy/security requirements must be re-reviewed and stronger infrastructure separation should be considered.

25.5 AI

AI assistance may support navigation, FAQs, search and operations. It must not become an uncontrolled diagnostic system. Any processing of sensitive health content requires explicit product, privacy and safety design before activation.

26. Primary Production Workflows

Patient appointment

1. Visitor searches clinic directory.
2. Visitor opens verified clinic profile.
3. Patient authenticates/registers if required.
4. Patient selects service/dentist (optional), date/time and note.
5. Appointment transaction is created as Requested.
6. After commit, notification jobs are queued.
7. Clinic sees request and confirms/reschedules/declines.
8. Patient sees updated status and receives configured notifications.
9. Clinic marks Completed/No-show where applicable.
10. Eligible patient can submit review.

Clinic verification

11. Clinic administrator creates account.

12. Clinic completes profile and uploads required documents.
13. Submission status becomes Submitted/Under Review.
14. Authorized admin reviews metadata and protected files.
15. Admin approves, rejects or requests changes with note.
16. Audit record is written.
17. Clinic receives in-app/external notification asynchronously.
18. Approved clinic becomes eligible for verified public listing.

Supplier RFQ

19. Clinic browses marketplace.
20. Clinic opens product and supplier.
21. Clinic submits quantity/message RFQ.
22. RFQ commits and notification job is queued.
23. Supplier views RFQ and responds/updates status.
24. Clinic sees response/status.
25. Admin can monitor/moderate if complaint is raised.

Admin moderation

26. Admin authenticates with MFA.
27. Admin opens scoped moderation queue.
28. Admin reviews entity and relevant evidence.
29. Admin performs permitted action.
30. State transition is validated.
31. Audit entry records action.
32. Related cache is invalidated and notification queued.

27. Analytics & Reporting Design

- Analytics may never become a hard dependency for patient, appointment or marketplace transactions.
- Transactional tables remain the source of truth; aggregated daily_metrics tables support dashboards.
- Event capture is lightweight and queued when possible.
- Dashboard filters use bounded date ranges and indexed scope columns.
- Exports over large ranges run in background jobs.
- No analytics event should contain raw passwords, document contents or unnecessary health data.

Area	Minimum MVP metrics
Patient	User registrations, clinic enrollments, clinic searches/views, appointment requests, status conversion and repeat appointments by clinic relationship.
Clinic	Registrations, verification completion, profile completion, enrolled patients, request volume, response/acceptance/completion.
Supplier	Registrations, verification completion, active products, product views, RFQ volume/status.
Marketplace	Search/category/product/supplier views, RFQ conversion.
Operations	Verification backlog, complaints, moderation activity, failed notifications/jobs, active entities.

Use the approved Dental Connect visual direction and attached healthcare references as visual guidance; implement natively, not as screenshots.

Primary visual language: modern healthcare-tech with white/off-white surfaces, dark text, teal/mint/aqua accents, selective soft gradients and restrained glassmorphism/frosted panels.

- Landing is a simple responsive public website. Patient experience is mobile-app-first and app-like; clinic, supplier and admin experiences are responsive professional dashboards.
- Patient mobile uses bottom navigation without a marketplace tab. Clinic/supplier/admin mobile use compact headers/drawers; marketplace is visible only to clinic, supplier and authorized admin roles.

- Glassmorphism is decorative, not structural: text contrast, form readability, performance and accessibility take precedence over transparency/blur.
- Every data view has loading, empty, error, offline/integration-unavailable and permission-denied states.
- Accessibility target: WCAG 2.1 AA practices for contrast, labels, keyboard/focus and semantic structure.
- Every data view has loading, empty, error and permission-denied states.
- Accessibility target: WCAG 2.1 AA practices for contrast, labels, keyboard/focus and semantic structure.

29. Approved Screen Inventory

The approved mockup manifest contains 66 unique screens and 132 desktop/mobile image references. The production implementation uses responsive routes/components rather than separate applications for each viewport.

Side	Unique screens	Responsive outputs
Public_Patient	15	30
Clinic	17	34
Restricted_Marketplace	5	10
Supplier	11	22
Admin	21	42

Side	Code	Screen
Admin	A01	Admin Login
Admin	A02	Central Dashboard
Admin	A03	Clinic Verification Queue
Admin	A04	Clinic Verification Detail
Admin	A05	Supplier Verification Queue
Admin	A06	Supplier Verification Detail
Admin	A07	Patient Management
Admin	A08	Clinic Management
Admin	A09	Supplier Management
Admin	A10	Dentist Management
Admin	A11	Appointment Monitoring
Admin	A12	Product / Category Moderation
Admin	A13	RFQ / Order Enquiry Monitoring
Admin	A14	Reviews Moderation
Admin	A15	Complaints / Disputes
Admin	A16	Commissions Readiness / Settings
Admin	A17	Analytics / Reports
Admin	A18	Roles / Permissions
Admin	A19	Audit Logs
Admin	A20	Notifications / Integration Settings
Admin	A21	Platform Settings
Clinic	C01	Clinic Registration
Clinic	C02	Clinic Onboarding / Profile Setup
Clinic	C03	Verification Documents
Clinic	C04	Verification Status / Changes Requested
Clinic	C05	Clinic Dashboard

Side	Code	Screen
Clinic	C06	Public Profile Editor
Clinic	C07	Services / Specialties
Clinic	C08	Dentists List
Clinic	C09	Add / Edit Dentist
Clinic	C10	Availability / Schedule Settings
Clinic	C11	Appointment List / Calendar
Clinic	C12	Appointment Detail / Actions
Clinic	C13	Clinic Analytics
Clinic	C14	Marketplace Entry / Browse
Clinic	C15	Clinic Account / Settings
Clinic	C16	My Patients List
Clinic	C17	Clinic Patient Detail / Appointments / Records / Notes
Restricted_Marketplace	M01	Marketplace Home / Categories
Restricted_Marketplace	M02	Product Search / Results
Restricted_Marketplace	M03	Product Detail
Restricted_Marketplace	M04	Supplier Public Profile
Restricted_Marketplace	M05	Request Quotation / Enquiry
Public_Patient	P01	Landing / Home
Public_Patient	P02	Clinic Search / Results
Public_Patient	P03	Clinic Profile
Public_Patient	P04	Patient Registration
Public_Patient	P05	Patient Login
Public_Patient	P06	Forgot / Reset Password
Public_Patient	P07	Patient Dashboard
Public_Patient	P08	Appointment Request
Public_Patient	P09	Appointment Confirmation / Status
Public_Patient	P10	My Appointments
Public_Patient	P11	Appointment Detail / Reschedule / Cancel
Public_Patient	P12	Patient Profile / Settings
Public_Patient	P13	Review / Rating Submission
Public_Patient	P14	Patient Notifications / Messages
Public_Patient	P15	Clinic Selection / Enrollment / Active Clinic Switcher
Supplier	S01	Supplier Registration
Supplier	S02	Supplier Onboarding / Profile
Supplier	S03	Verification Documents / Status
Supplier	S04	Supplier Dashboard
Supplier	S05	Supplier Profile Editor
Supplier	S06	Product List

Side	Code	Screen
Supplier	S07	Add / Edit Product
Supplier	S08	RFQ / Enquiries List
Supplier	S09	RFQ / Enquiry Detail
Supplier	S10	Supplier Analytics / Basic Activity
Supplier	S11	Supplier Account / Settings

30. Production Delivery Milestones

Milestone	Deliverables / engineering gate
1. Architecture & Data	Final module map, ERD/data dictionary, indexes, route map, roles/permissions, hosting plan and risk register.
2. Foundation	Laravel app, modules, auth/RBAC, settings, files, audit, API foundation, CI/deployment baseline.
3. Patient/Public	Directory, clinic profile, patient identity, clinic selection/enrollment, clinic-scoped patient app, appointment request/status and responsive UI.
4. Clinic	Onboarding, verification, profile/services/dentists/availability, My Patients, patient detail, appointment management, analytics and restricted marketplace.
5. Supplier/Marketplace	Supplier verification, products/categories, restricted marketplace search/detail, supplier marketplace view and RFQ workflow.
6. Administration	Verification queues, users, moderation, reviews/complaints, permissions, audit/reporting.
7. Notifications & Hardening	Adapters, queues, retries, caching, query/index review, security testing, performance testing.
8. Deployment & Handover	Production deployment, backups, restore proof, documentation, company account access and UAT sign-off.

Patient can register/login, select or enroll with a verified clinic, use a clinic-scoped patient app, request appointments and see real status without access to marketplace.

Clinic can register, submit verification, manage approved profile/services/dentists/availability, manage its own patient list and patient details, manage appointment requests, and access the restricted marketplace.

- Patient can register/login, find verified clinics, view profiles, request appointments and see real status.
- Clinic can register, submit verification, manage approved profile/services/dentists/availability and manage requests.
- Supplier can register, verify, manage products and receive/manage RFQs.
- Admin can govern clinics, suppliers, users, products, appointments, reviews, complaints, roles and audit data according to permission.
- Data is real production persistence; no hard-coded dashboard values.
- Core lists are paginated and indexed; query plan review completed.
- No cross-organization or cross-patient data leakage.
- Notification provider failure does not lose core business transactions.
- Backups exist and restore is proven.
- Production is HTTPS-only, debug disabled, secrets protected and admin MFA active.
- Company owns source repository and production accounts.
- Critical desktop/mobile UAT is approved.

32. Ownership, IP, Accounts & Handover

- Project-specific source code, frontend, backend, database schema/migrations, API code, UI/UX, documentation, architecture and custom scripts are handed over according to the signed agreement.
- Production domain, hosting, repository, database, messaging/payment provider accounts and credentials are company-controlled wherever the provider allows.
- No production dependency may exist only inside a developer personal account.
- Open-source dependencies and licenses are documented.
- Handover includes setup/deployment guide, environment-variable reference, database/data dictionary, backup/restore procedure and operational runbook.
- Another competent Laravel developer must be able to continue the project without proprietary lock-in.

33. Risks, Assumptions & Open Product Decisions

Topic	Current position / decision needed
Brand name	Dental Connect is the approved product name. Production domain: dentalconnect.co.tz.
Patient login identifier	Confirm whether phone, email or both are mandatory and whether OTP is needed in MVP. One platform login identity may link to multiple separate clinic-patient enrollments.
Verification documents	Client/dental/legal owner must define exact clinic and supplier document requirements and expiry rules.
Appointment rules	Confirm dentist-specific booking, lead time, duration defaults and cancellation rules.
Reviews	Confirm publication/moderation policy and whether only completed appointments may review.
WhatsApp/SMS provider	Select provider/account before integration; costs and availability are third-party dependencies.
Hosting plan	Commercial proposal allocates initial hosting separately; exact Hostinger tier selected at purchase based on then-current pricing and resource limits.
Data protection	Client should confirm PDPC registration/compliance obligations and final privacy/retention policies with appropriate legal/privacy advice.
Future clinical records	Separate security/compliance architecture review required before development.
Patient ownership model	Approved: patient operational records belong to the clinic through clinic_patients. A person enrolling at another clinic receives a separate clinic-patient record even when the same platform login identity is reused.
Marketplace access	Approved: marketplace is restricted to clinic users, supplier users and authorized platform administrators. Guests and patient accounts have no marketplace access.

34. Technical Validation Notes (September 2026)

These notes document the external facts used to keep the stack compatible with the intended initial host. They are implementation references, not commercial guarantees.

Reference	Validated fact used in architecture
Hostinger database platform	Hostinger states that Web and Cloud hosting use MariaDB and that InnoDB is supported. PostgreSQL and Redis require VPS hosting.
Hostinger Laravel/PHP	Hostinger supports Laravel on Web/Cloud hosting and PHP 8.2+; PHP 8.3 is a current default option and newer supported versions are selectable depending on plan.
Hostinger cron/Composer	Web/Cloud plans support cron jobs and Composer/SSH workflows suitable for Laravel scheduler and deployment.
Laravel support	Laravel 13 was released March 17, 2026 and supports PHP 8.3-8.5, with security fixes scheduled through March 17, 2028.
Tanzania privacy	The Personal Data Protection Commission publishes the Personal Data Protection Act, 2022; the regulator identifies health information

Reference	Validated fact used in architecture
	as sensitive personal data.

Reference URLs (human-readable): hostinger.com/support/1583226-which-database-management-system-is-used-at-hostinger/ ; hostinger.com/support/which-databases-and-data-tools-are-supported-at-hostinger/ ; hostinger.com/support/which-programming-languages-and-frameworks-are-supported-at-hostinger/ ; hostinger.com/support/1583465-how-to-set-up-a-cron-job-at-hostinger/ ; laravel.com/framework/docs/releases ; pdpc.go.tz/the-personal-data-protection-act-2022/ .

35. Launch Readiness Checklist

- Production domain/DNS and SSL verified.
- PHP version, OPcache and production environment verified.
- MariaDB migrations executed and index review complete.
- Admin MFA enabled; default/demo credentials removed.
- Roles/permissions and organization-scope negative tests pass.
- Private upload access controls tested.
- Notification provider credentials verified or integration explicitly marked unavailable with graceful fallback.
- Scheduler/queue cron tested and failed-job monitoring enabled.
- Backups configured; clean-environment restore test passed.
- Application debug disabled and secrets absent from repository/logs.
- Performance tests pass or documented exceptions approved.
- Responsive patient, clinic, supplier and admin critical journeys approved.
- Company repository/hosting/database/admin credentials handed to authorized owner.
- Incident/rollback contacts and process documented.

This architecture is deliberately conservative: it prioritizes secure clinic-owned patient relationships, restricted business-only marketplace access, fast indexed SQL, low front-end overhead, recoverable module failures, straightforward deployment and maintainability. It gives Dental Connect a credible path to mobile apps, payments, ordering, logistics, financing, insurance, cloud dental records and regional growth without forcing those future costs into the initial system.

Build decision

Use Laravel 13 on PHP 8.3/8.4 with Blade + Livewire + Alpine + Tailwind, MariaDB/InnoDB, a versioned REST API and a modular-monolith domain structure. Start on Hostinger Web/Cloud without requiring Redis. Use database queues and cron, then move cache/queues and selected modules to VPS/Redis/services only when product usage justifies the operational complexity.

This architecture is deliberately conservative: it prioritizes secure data ownership, fast indexed SQL, low front-end overhead, recoverable module failures, straightforward deployment and maintainability. It gives the MVP a credible path to mobile apps, payments, ordering, logistics, financing, insurance, cloud dental records and regional growth without forcing those future costs into the initial system.

Appendix A. Core Non-Functional Requirement Register

ID	Priority	Requirement
NFR-PERF-001	P0	All high-volume lists paginated server-side.
NFR-PERF-002	P0	No unresolved N+1 queries in critical routes.
NFR-PERF-003	P0	Core hot-table query plans/indexes reviewed before launch.
NFR-PERF-004	P1	Analytics uses aggregates/cache for repeated heavy metrics.
NFR-SEC-001	P0	HTTPS-only production and secure

ID	Priority	Requirement
		sessions.
NFR-SEC-002	P0	Server-side RBAC and organization scope on every protected resource.
NFR-SEC-003	P0	Admin MFA.
NFR-SEC-004	P0	Private verification-file access control.
NFR-SEC-005	P0	Secrets excluded from source and client responses.
NFR-REL-001	P0	External provider outage cannot break unrelated core modules.
NFR-REL-002	P0	Critical writes are transactional and notification side-effects after commit.
NFR-REL-003	P0	No blank-page failure on major route.
NFR-REL-004	P1	Health/failed-job/backup monitoring enabled.
NFR-DATA-001	P0	MariaDB/InnoDB schema with foreign keys and documented ownership.
NFR-DATA-002	P0	BIGINT internal PK + opaque public ULID on exposed entities.
NFR-DATA-003	P0	UTC timestamps and DECIMAL future monetary values.
NFR-DATA-004	P0	Daily backup and restore proof.
NFR-OPS-001	P0	Development/staging/production separated.
NFR-OPS-002	P0	Company-controlled repository and production accounts.
NFR-OPS-003	P0	Debug disabled and smoke test after deployment.

Appendix B. Engineering Rules for Database Review

- Every new list/search endpoint documents expected filters, sort order, pagination and indexes.
- Every new table documents module owner, retention class, sensitivity class and expected growth pattern.
- Every migration adding an index to a large table is rehearsed in staging.
- Every complex ORM query is inspected for generated SQL in development/staging.
- Use EXPLAIN on slow or high-frequency queries; compare rows examined vs rows returned.
- Do not add indexes speculatively without considering write cost and duplicate-prefix indexes.
- Review redundant indexes quarterly as the dataset grows.
- Archive or aggregate high-growth logs/events instead of allowing unlimited hot-table growth.
- Keep status/date/scope columns used together in composite indexes in the same order as real query predicates and sorting, validated empirically.